

Eclipse IDE Cheat Sheet

Beginner's guide on navigating through Eclipse



By Alizah Saqib and Deb Harding

Contents

3. [Eclipse IDE](#)
4. [Create Project](#)
5. [Create Class and Java File](#)
6. [Navigating Java Perspective](#)
7. [Edit, Run and Compile](#)
8. [Add Existing Java File with Drag and Drop](#)
9. [Errors](#)
10. [Warnings](#)
11. [Reset Perspective and Views](#)
12. [Formatter Profile Set Up](#)
13. [Auto-Format Save](#)
14. [Change Themes, Colors Fonts](#)
15. [Debug - Set Breakpoints](#)
16. [Debug - Start Debug Session](#)
17. [Debug - Step Through Code](#)
18. [Generate Comments](#)
19. [Keyboard Short Cuts](#)
20. [More Support](#)



Eclipse IDE

Different IDEs can be used for different languages and use cases. [10 Best Java IDE For Developers in 2025 - GeeksforGeeks](#)

Why and IDE?

- Makes coding easier and faster
- Helps you avoid mistakes
- Organizes your project files in one place
- Gives you immediate feedback while you work

Why Eclipse?

- Free & Reliable — No cost, works across Windows, macOS, and Linux.
- Industry-Standard Tool — Still widely used in Java development.
- Java-Focused Features — Offers auto-completion, auto-formatting, debugging, and more to make coding smoother.
- Real-World Learning — Helps you practice organization, testing, and debugging in a professional environment.

First set up [Git and Github](#) and then [Setup Eclipse](#) on Windows or Mac OS



1. On the top menu, select **File->New-> Java Project** to display the New project dialog

Create Project JDK-25

The screenshot shows the 'New Java Project' dialog box in the Eclipse IDE. The dialog is titled 'New Java Project' and contains several sections for configuring a new project. Annotations with arrows point to specific fields and options:

- Project name:** The text 'ProjectName' is entered in the field. An annotation says: 'Give the project a name **FirstProject** Do not use any spaces'.
- Location:** The path '/Users/debteach/Documents/GitHubRepos/CS1050_Fall2024/EclipseWorkSpace/ProjectName' is entered. An annotation says: 'This should already be set to the EclipseWorkSpace in your GitHubRepos/CSJavaRepos folder'.
- JRE:** Under 'Use an execution environment JRE:', 'JavaSE-24' is selected. An annotation says: 'This should be the latest version you installed JavaSE25 or some might need to select Default depending on how you set it up.'.
- Project layout:** Under 'Use project folder as root for sources and class files', the radio button is selected. An annotation says: 'Select use project folder'.
- Working sets:** The 'Add project to working sets' checkbox is unchecked.
- Module:** The 'Create module-info.java file' checkbox is checked. The 'Module name' field is empty. The 'Generate comments' checkbox is also checked.

At the bottom of the dialog, there are buttons for '< Back', 'Next >', 'Cancel', and 'Finish'.

This should already be set to the EclipseWorkSpace in your GitHubRepos/CSJavaRepos folder

This should be the latest version you installed JavaSE25 or some might need to select Default depending on how you set it up.

Select use project folder

Make sure all settings are correct. Click Finish.

Create Class & Java File

1. Select the project folder you want to create it in - in the package explorer on the left, click on your project to highlight it
2. Create a new class - click file > new > class
3. Configure settings -
 - a. Source folder should be set, if not, select the project folder you want it to go in
 - b. Package should be empty, if not, delete the package
 - c. Enter a name for your class (e.g., GEM01Calculation)
 - d. Modifiers: checkmark "public"
 - e. Method stubs: checkmark "public static void main(String[] args)"
 - f. Comments: checkmark "generate comments"
4. Click finish to create the class
5. This create your java file with class and main method

Step 1

Step 2

Step 3a

Step 3b

Step 3c

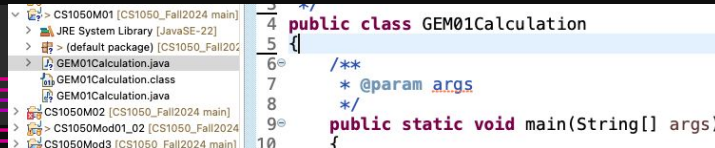
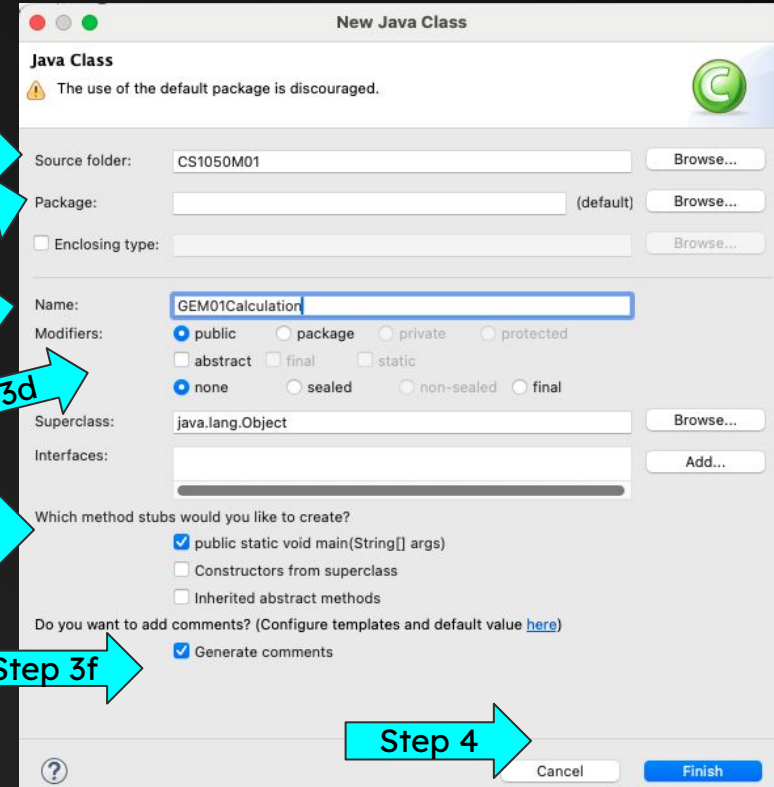
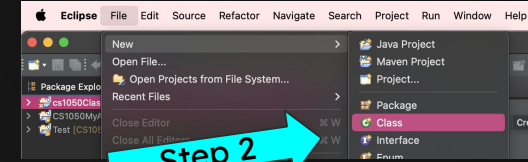
Step 3d

Step 3e

Step 3f

Step 5

Step 4



Navigating Java Perspective

Running your code

The screenshot shows the Eclipse IDE in the Java perspective. The interface is divided into several panes: the Project Explorer on the left, the central editor showing the code for GEM01Calculation.java, and the Console at the bottom. Annotations with arrows point to various components: 'Menus' points to the top menu bar; 'Run Program' points to the Run button in the toolbar; 'Toolbar' points to the toolbar itself; 'Project Explore' points to the Project Explorer; 'Project Folders' points to the project hierarchy in the Project Explorer; 'Project contain JRE System and java files' points to the 'GEM01Calculation.java' file in the Project Explorer; 'Edit code' points to the code editor; 'Console Tab' points to the Console tab at the bottom; and 'Console display' points to the output text 'Hello Java' in the Console.

Annotations and their targets:

- Menus**: Points to the top menu bar (File, Edit, etc.).
- Run Program**: Points to the Run button (a green play icon) in the toolbar.
- Toolbar**: Points to the toolbar containing various icons for file operations, editing, and running.
- Project Explore**: Points to the Project Explorer pane on the left.
- Project Folders**: Points to the project hierarchy in the Project Explorer.
- Project contain JRE System and java files**: Points to the 'GEM01Calculation.java' file in the Project Explorer.
- Edit code**: Points to the code editor showing the Java code for GEM01Calculation.
- Console Tab**: Points to the Console tab at the bottom of the IDE.
- Console display**: Points to the output text 'Hello Java' in the Console.

Code shown in the editor:

```
1 /**
2  *
3  */
4 public class GEM01Calculation
5 {
6     /**
7      * @param args
8      */
9     public static void main(String[] args)
10    {
11        // print method
12        System.out.println("Hello Java");
13    }
14 }
15 }
16 }
17 }
```

Console output:

```
<terminated> GEM01Calculation [Java Application] /Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java (Aug 11, 2025, 6:02:49 PM - 6:02:50 PM) [pid:
Hello Java
```

Edit, Run and Compile



When you click Run (▶) in Eclipse, two things happen:

1. **Save & Compile** – Eclipse automatically saves your changes and compiles your .java source files into .class files (Java bytecode).
2. **Run on the JVM** – The compiled .class files are then executed by the Java Virtual Machine (JVM).

Compilation checks your code for syntax errors and turns it into a format the JVM can understand. If there are errors, Eclipse won't run the program until they're fixed.

Warnings don't stop compilation, but they should still be fixed to avoid future problems.

Editing Code:

- You can change your code anytime in the editor.
 - When you run the program again, Eclipse automatically recompiles the changed files.
 - You don't have to manually compile in most cases—Eclipse handles it for you.
- 

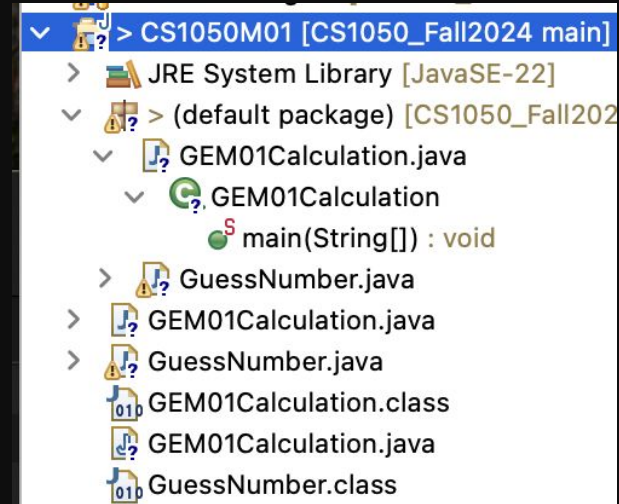
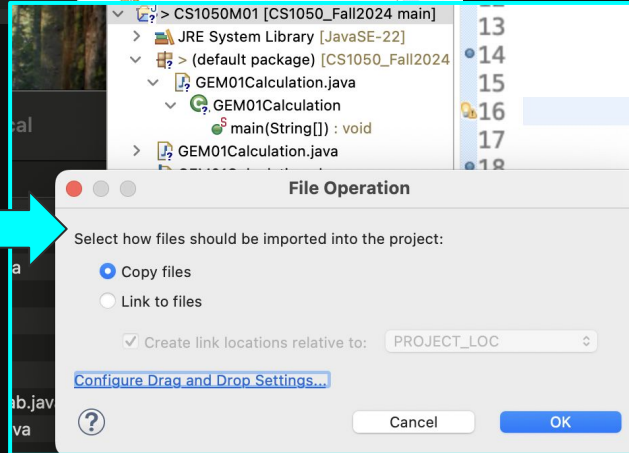
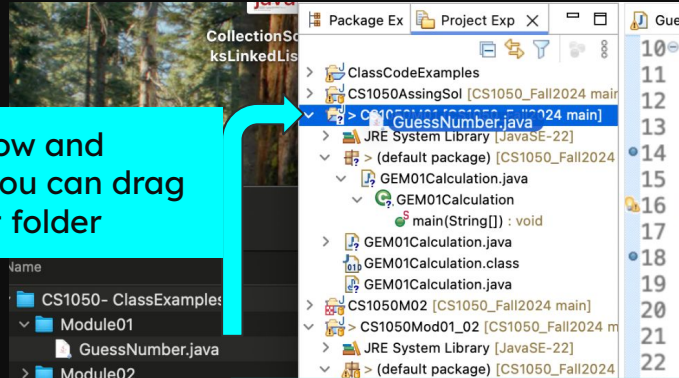
Add Existing Java File with Drag and Drop

You can add an existing Java file to a current project.

1. Make folder window and Eclipse window so you can drag java file into project folder

2. Click copy files so it will put the file in your repo workspace folder for versioning.

You should see the .java and .class file in the project. If you go to your github desktop you can commit and push a new version.



Errors

Error

Error Location - hover over for information on how to fix.

Problem

Problems Tab

Shows errors from all files.

Error Information

Description

File name

Line Number

Console Tab

Program won't run and displays error in Console window

The screenshot shows an IDE with a Java file named `GuessNumber.java`. The code is as follows:

```
9 //Class - Every program must have a class where there is a starting b
10 //ending brace. The ending brace is a the end of the file
11 public class GuessNumber
12 {
13
14     // Main Method - the main method is in the class to perform the a
15     // and this is where the code goes between the starting and endin
16     public static void main(String[] args)
17     {
18         Syntax error, insert ";" to complete BlockStatements
19         int number = (int) (Math.random() * RANDOM_MULTIPLIER + 1;
20
21         Scanner keyboardInput = new Scanner(System.in);
22
23         System.out.println("Guess a magic number between 0 and 100");
24     }
25 }
```

The **Problems** tab shows the following error:

Description	Resource	Path	Location	Type
Syntax error, insert ";" to complete BlockStatements	GuessNumber.java	/CS1050Mod01_02	line 18	Java Problem

The **Console** tab shows the following error message:

```
<terminated> GuessNumber (2) [Java Application] /Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java (Aug 11, 2025, 6:09:22 PM - 6:09:22)
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    Syntax error, insert ";" to complete BlockStatements

    at GuessNumber.main(GuessNumber.java:18)
```

Warnings

Your program should not contain warnings. Why fix them?

- Possible Security Risks – Some warnings indicate unsafe code that could be exploited.
- Maintainability – Clean code is easier for you (and others) to read, debug, and improve.
- Professional Standards – In industry, code reviews often require zero warnings before approval.
- Prevent Future Bugs – Warnings can signal logic issues or outdated methods that might cause errors later.

The screenshot shows an IDE with a code editor on the left and a Problems tab on the right. The code editor displays a Java program with a warning: "Resource leak: 'keyboardInput' is never closed". The Problems tab shows a list of warnings, including "Resource leak: 'input' is never closed" and "Resource leak: 'keyboardInput' is never closed".

warning →

Warning Location - hover over for information on how to fix. →

Problem →

Problems Tab →

Shows warnings from all files →

Warning Information →

Description	File name	Path	Location
Resource leak: 'input' is never closed	M02L06MathChars.java	/ClassCodeExamples	line 35
Resource leak: 'input' is never closed	project01iteration.java	/ClassCodeExamples	line 71
Resource leak: 'keyboardInput' is never closed	GuessNumber.java	/CS1050Mod01_02	line 22

Description

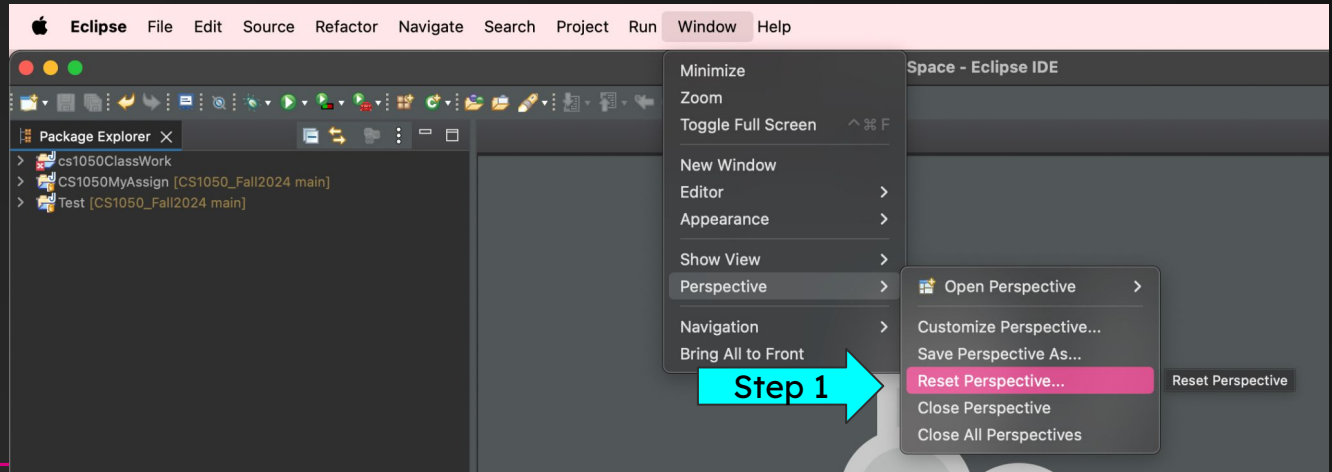
File name

Line Number

Reset Perspective and Views

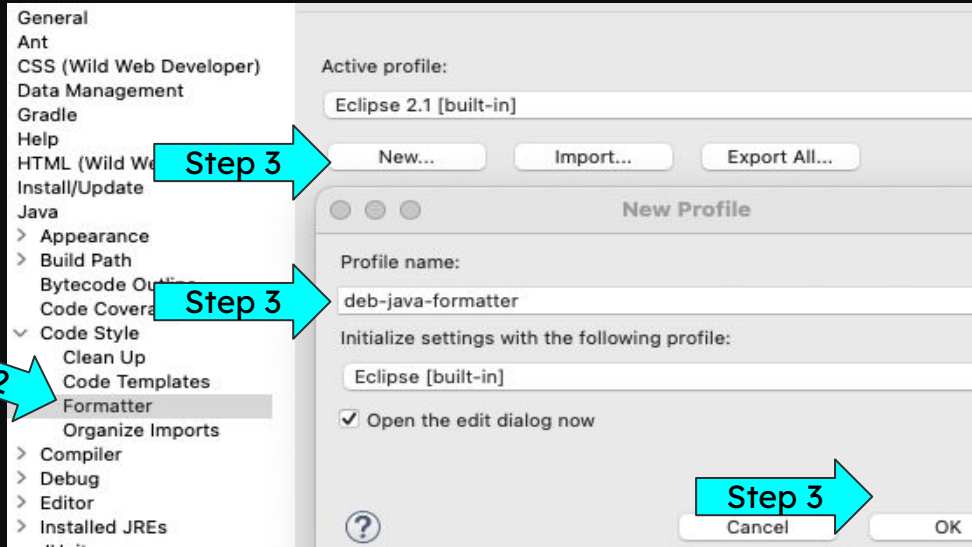
A “perspective” is just a layout of views and controls what you see in certain menus and toolbar. You will be using the Java and Debug Perspectives

1. If your “perspective” gets messed up Click window > perspective > reset perspective
 - a. If you accidentally close a panel and want to bring it back
 - b. If important windows like the console or package explorer disappear

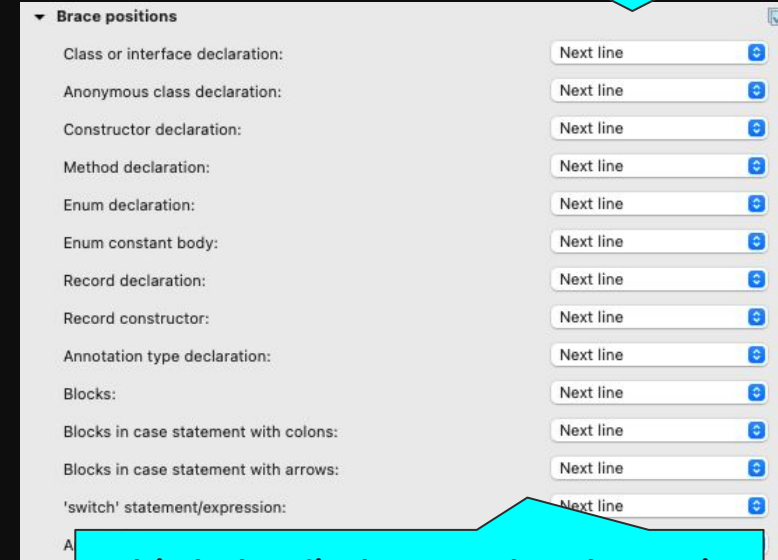


Formatter Profile Set Up

1. Go to settings (or might be called preferences)
 - a. Windows OS Go to menus Window -> preferences
 - b. Mac OS go to menus Eclipse -> settings
2. Then navigate to Java -> Code Style -> Format
3. Click new, Give your profile a name and click ok



4. Go to the Braces section, for all categories set Position to Next line.
5. Click ok to save profile.
6. Click apply and save



This helps find error when brace is missing or incorrectly placed.

Auto-Format on Save

Save Actions on, running code will always auto-format since Eclipse saves before compiling.

1. Go to
 - a. Mac: Eclipse → Settings → Java → Editor → Save Actions
 - b. Windows: Window → Preferences → Java → Editor → Save Actions
2. Select these save actions



Step 1

Save Actions

Step 2

- ☒ Perform the selected actions on save
- ☒ Format source code
 - ☒ Format all lines
 - ☐ Format edited lines

Configure the formatter settings on the [Formatter](#) page.
- ☒ Organize imports
 - Configure the organize imports settings on the [Organize Imports](#) page.
- ☐ Additional actions
 - Convert control statement bodies to block
 - Add missing '@Override' annotations
 - Add missing '@Override' annotations to implementations of interface methods
 - Add missing '@Deprecated' annotations
 - Add necessary casts

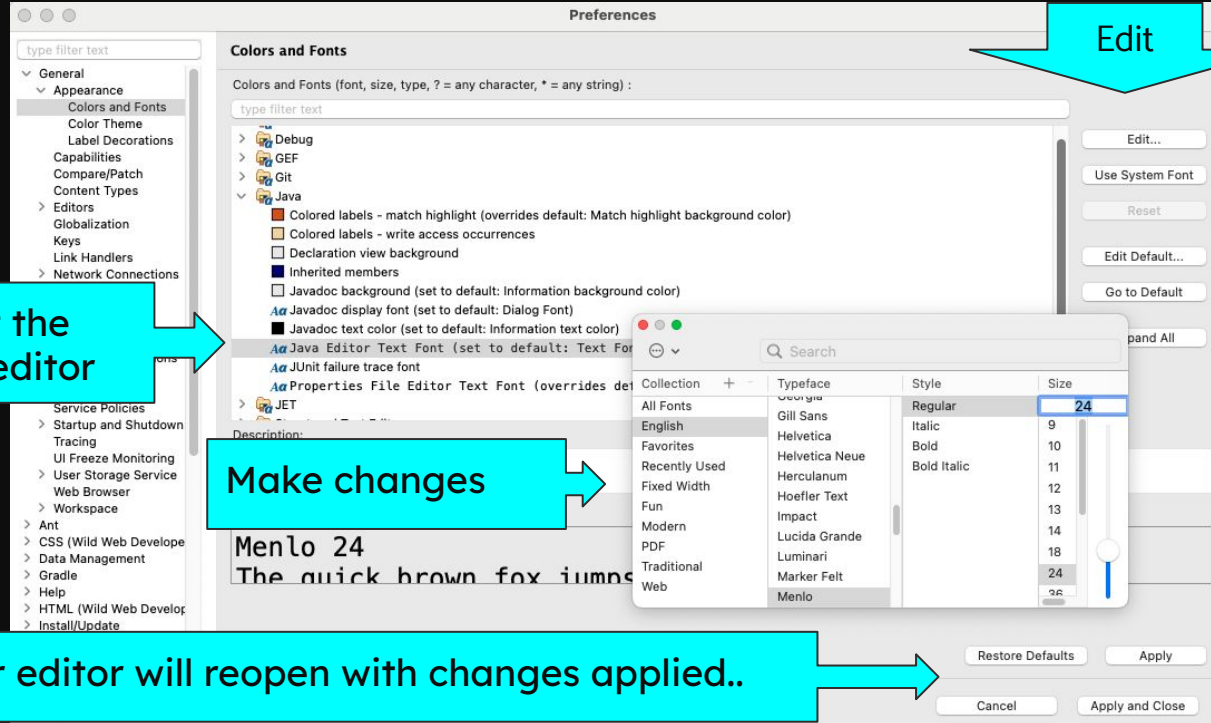
Type a messy block of code, then click Run — you should see the the correct format.

Change Themes, Colors Fonts

[6 Best Fonts for Coding to Keep Your Eyes from Eyestrain | by plabs.id](#)

If you want change the defaults go to Settings/Preferences general-> Appearance ->

- Colors and Fonts:
- Color Theme



For example, you can edit the font and size for the text editor

Make changes

Apply and your editor will reopen with changes applied..

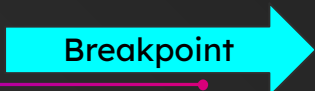
Debug - Set Breakpoints

Set Breakpoints to step through each statement and see what is stored in each variables memory

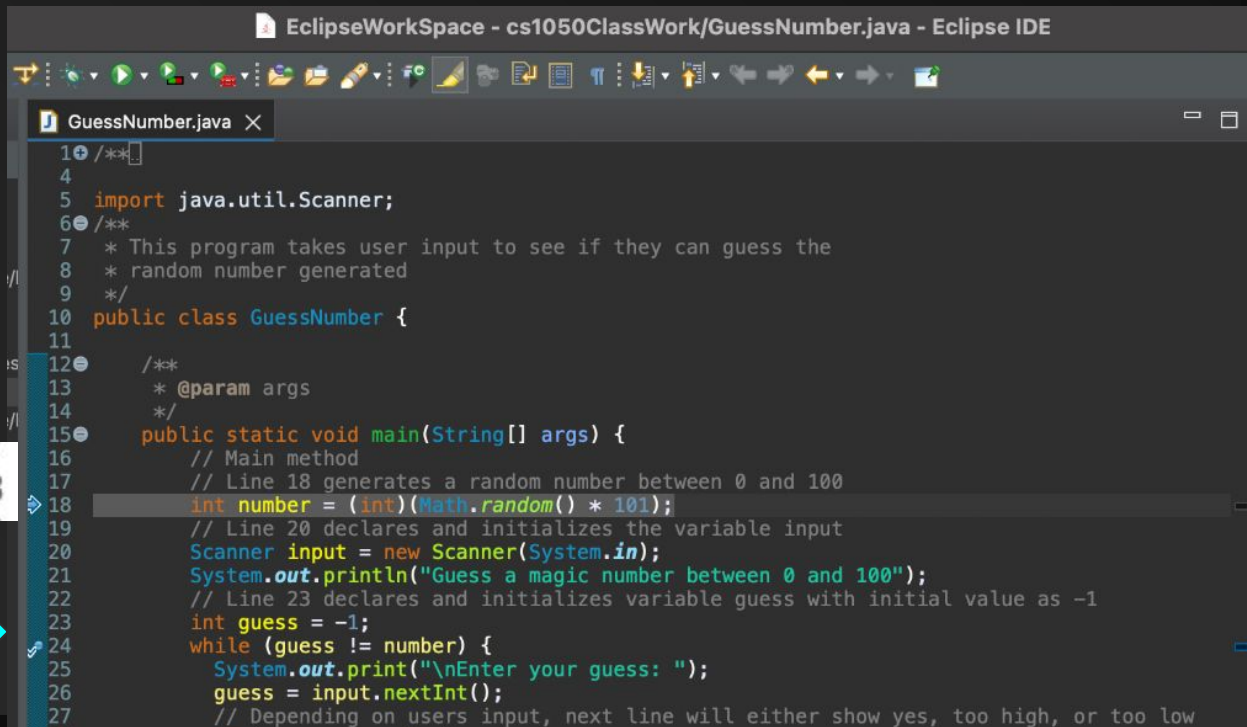
- To set a breakpoint, double click directly to the left of the line numbers
- A blue circle will appear next to the line, indicating that a breakpoint has been set
- To get rid of the breakpoint, double-click on it and the blue dot will disappear



Breakpoint



Breakpoint



```
1 /**
4
5 import java.util.Scanner;
6 /**
7  * This program takes user input to see if they can guess the
8  * random number generated
9  */
10 public class GuessNumber {
11
12     /**
13     * @param args
14     */
15     public static void main(String[] args) {
16         // Main method
17         // Line 18 generates a random number between 0 and 100
18         int number = (int)(Math.random() * 101);
19         // Line 20 declares and initializes the variable input
20         Scanner input = new Scanner(System.in);
21         System.out.println("Guess a magic number between 0 and 100");
22         // Line 23 declares and initializes variable guess with initial value as -1
23         int guess = -1;
24         while (guess != number) {
25             System.out.print("\nEnter your guess: ");
26             guess = input.nextInt();
27             // Depending on users input, next line will either show yes, too high, or too low
```

Debug: Start Debugging Session

To start a debugging session click the debug icon (a green bug) in the toolbar before the run button. You should see the variables window (Debug Perspective). If you don't this could be because

- You didn't create your code in a project
- The java code has syntax errors

Debug window

Variables window

Run **Window** **Help**

- ▶ Resume
- ⏸ Suspend
- ⏹ Terminate

Breakpoint Types

- Toggle Breakpoint
- Toggle Lambda Entry Breakpoint
- Toggle Tracepoint
- Toggle Line Breakpoint
- Toggle Watchpoint
- Toggle Method Breakpoint
- Skip All Breakpoints

GuessNumber.java

```
10 /**
11  *
12  * @author
13  */
14 public class GuessNumber {
15     // Main method
16     // Line 18 generates a random number between 0 and 100
17     int number = (int)(Math.random() * 101);
18     // Line 20 declares and initializes the variable input
19     Scanner input = new Scanner(System.in);
20     System.out.println("Guess a magic number between 0 and 100");
21     // Line 23 declares and initializes variable guess with initial value as -1
22     int guess = -1;
23     while (guess != number) {
24         System.out.print("\nEnter your guess: ");
25     }
```

Variables

Name	Value
no method return value	
args	String[] (id=19)

Console

GuessNumber [Java Application] /Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java (Mar 6, 2025, 9:30:07 PM elapsed: 0:00:47) [pid: 57759]

Do not select Skip All Breakpoints

If the program didn't stop you may not have set breakpoints or it is skipping breakpoints. Make sure Skip All Breakpoints is not turned on in Run menu.

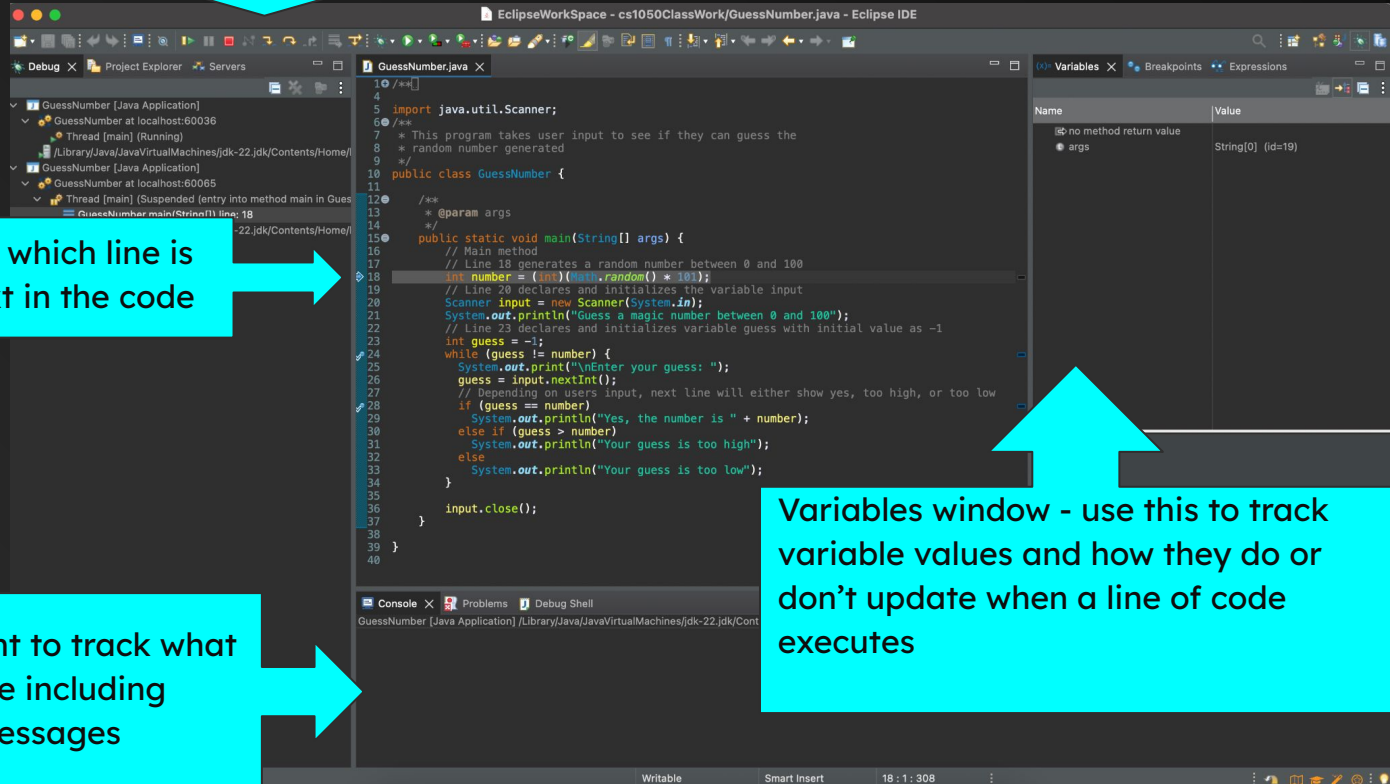
Debug: Step Through Code

Use the “step over” button to control the flow of the program and move through each line manually.

Menus



The pointer tells us which line is to be executed next in the code

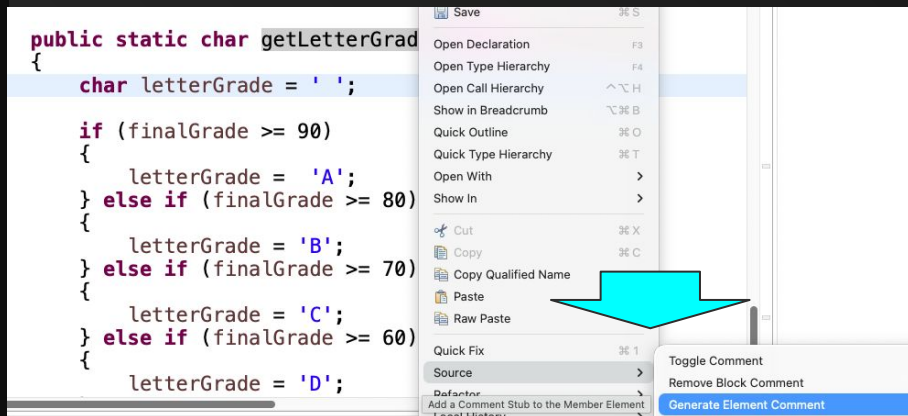


Variables window - use this to track variable values and how they do or don't update when a line of code executes

Console output - it's important to track what is being printed to the console including print statements and error messages

Generate Comments

Right Click inside a method and select Source -> Generate Element comment



/**

element comment

* @param

finalGrade

* @return

*/

public static char

getLetterGrade(double

finalGrade)

{

char

letterGrade = ' ';

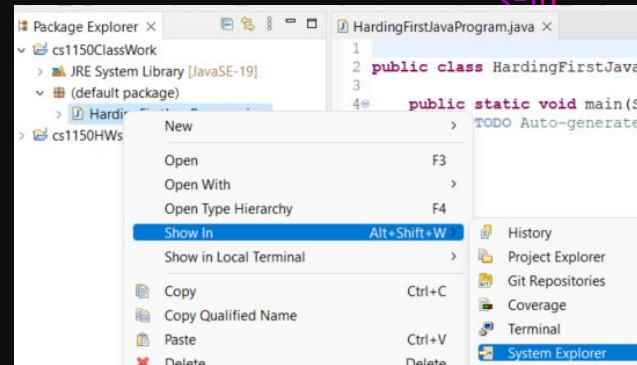
Keyboard Shortcuts

Action	Mac	Windows
Run Program	⌘ + F11	Ctrl + F11
Debug Program	F11	F11
Format Code	⌘ + ⇧ + F	Ctrl + Shift + F
Organize Imports	⌘ + ⇧ + O	Ctrl + Shift + O
Search / Find	⌘ + F	Ctrl + F
Find in Project	⌘ + H	Ctrl + H
Open Type (Class)	⌘ + ⇧ + T	Ctrl + Shift + T
Open Resource (File)	⌘ + ⇧ + R	Ctrl + Shift + R
Rename Variable/Method/Class	Alt + ⌘ + R	Alt + Shift + R
Switch Tabs	⌘ + Option + → / ←	Ctrl + PageDown / PageUp
Close Tab	⌘ + W	Ctrl + W
Undo	⌘ + Z	Ctrl + Z
Redo	⌘ + ⇧ + Z	Ctrl + Y

More Support

You can navigate to the file in your system explorer

- right click the file then go to Show In -> System Explorer



More Eclipse Resources

- [Java Debugging with Eclipse - Tutorial](#)
- [Eclipse Help](#)

“ How does a Java dev
cut his hair?

Eclipse it ”

@nhazelton

